

Logistic Regression

Intro and Motivation

Logistic Regression is for working on **classification** problems. The simplest type is the **binary classifier**, $y \in \{0, 1\}$

Why not Linear Regression

Couldn't we train a linear model to predict $y \in \{0, 1\}$? There are two problems with this. A linear model can produce any value $(-\infty, \infty)$, which we obviously don't want. Secondly, the MSE loss treats the distance between predictions as meaningful in a way that does not apply to class labels.

What we want is a model that outputs a number between 0 and 1 that we can interpret as the **probability** that the input belongs to a given class.

Logits and Sigmoids

So, imagine we are trying to extend our idea of linear regression to classification. We know this doesn't work, since probabilities are in $[0, 1]$ and the output of a regression model would be in $(-\infty, \infty)$. The key idea is this:

What if we apply some **function** on $p \in [0, 1]$, to map it to all of $\mathbb{R}$? Then, we use regression to **predict the output of that function**. Finally, we perform the **inverse transformation** on the output to convert it back to a probability. Sounds like a plan, but what function should we use?

Smart people have found that the best function to use that maps $[0, 1] \mapsto (-\infty, \infty)$ is:

$$f(p) = \log\left(\frac{p}{1-p}\right)$$

Called the **logit**. They chose this function over other functions that perform the same mapping because it has a lot of **nice properties**:

- $f(p)$ is defined on $(0, 1)$
- $f(p)$ is symmetric around $f(0.5)$
- Swapping the class labels (swapping p and $1-p$) flips the function:
 $f(1-p) = -f(p)$

The use of the logarithm gives the logit a nice **additive structure** that lends itself well to regression.

When you have two **independent features** that each contribute to the probability separately, we know that their effects **multiply**.

$$P(X \text{ and } Y) = P(X)P(Y) \text{ if } X, Y \text{ independent}$$

The log turns the multiplication into **addition**, which we need for a linear $w^T x + b$ style model.

So, we solve the linear model in the logit space:

(augmented vectors, so bias included)

$$\log(p/(1-p)) = w^T x$$

We now need to **invert this** to solve for the **probability p** .

Solving for p :

$$p/(1-p) = e^{w^T x}$$

$$p = e^{w^T x} + p e^{w^T x}$$

$$p(1 - e^{w^T x}) = e^{w^T x}$$

$$p = \frac{e^{w^T x}}{1 + e^{w^T x}}$$

We have just discovered the sigmoid function $\sigma(x)$

$$\sigma(x) = \frac{e^x}{1 + e^x}$$

It maps an unbounded input to a probability.

The Model

Given an input $x \in \mathbb{R}^d$ and parameters $w \in \mathbb{R}^d, b \in \mathbb{R}$, logistic regression model predicts the probability that $y=1$ (the probability that the input belongs to class 1).

$$P(y=1 | x, w, b) = \sigma(w^T x + b)$$

or, after augmentation:

$$P(y=1 | x, w, b) = \sigma(\tilde{w}^T \tilde{x})$$

We choose 0.5 as our decision boundary e.g:

$$\hat{y} = 1 \text{ if } \sigma(w^T x) \geq 0.5 \text{ else } \hat{y} = 0$$

Since $\sigma(z) \geq 0.5$ iff $z \geq 0$, this is the same as:

$$\hat{y} = 1 \text{ if } w^T x \geq 0, \text{ else } \hat{y} = 0$$

The decision boundary is the set of points where $w^T x = 0$, a hyperplane in $\mathbb{R}^d$ passing through the origin (in augmented space). So the logistic model partitions the input space into two half-spaces with a hyperplane.

So, to train the model, we ask ourselves: how do we find w such that the model predicts accurately for all x ? We first need to define our loss function. First, we need to understand the concept of likelihood.

Probability vs Likelihood, Maximum Likelihood Estim.

The simplest probability distribution for a binary outcome is the Bernoulli Distribution

$$P(y) = p^y (1-p)^{(1-y)} \quad (*)$$

[It is just the Binomial distribution for a single trial]

We see that

$$P(y=1) = p$$

$$P(y=0) = 1-p$$

as expected for our classifier. However, there are two lenses through which we can interpret (*), probability and likelihood.

- Probability lens - p is fixed y varies

"I know this coin has a 70% chance of showing heads. If I flip it 10 times do I expect to see 7 heads?" We know the world, and are reasoning about the outcomes

- Likelihood lens - y is fixed (observed), p varies

"I've just observed this coin show 7 heads from 10 flips, what underlying

value of p most plausible generated this?" You know the outcome and are reasoning about the world.

Maximum Likelihood Estimation (MLE)

formalises the likelihood lens. Given the observations we've seen, we create $L(p) = p^y(1-p)^{1-y}$ as a function of p alone, differentiate and maximise.

(in the coin example, this gives $p=0.7$, as we expect). So the MLE is the value of p that makes the observed data most probable.

So to continue, we model each label $y^{(i)}$ as a Bernoulli Random Variable with success probability $p^{(i)} = \sigma(w^T x^{(i)})$. Since each data point is independent of the others, the likelihood of observing the entire dataset together is the product of the individual likelihoods:

$$L(w) = \prod_i p^{y^{(i)}} (1-p)^{(1-y^{(i)})}$$

We're now asking: "What weights w make the entire observed dataset most probable?"

In other words, what value of w maximises the likelihood of seeing this data? This smells like calculus. We're going to have to differentiate this, but the $\prod$ makes things tricky.

However, we can cleverly take $\log$ of both sides to turn the product into a sum!

$$\log(L(w)) = \sum_i (y^{(i)} \log(p^{(i)}) + (1-y^{(i)}) \log(1-p^{(i)}))$$

Now, recall that we want to maximise the log-likelihood. However, by convention, gradient descent is for minimising things. So we simply negate, so that we can minimise $-\log(L(w))$, which solves the same problem. We also divide by n to get the average. So our final loss function is:

$$\mathcal{L}(w) = -\frac{1}{n} \sum_i y^{(i)} \log(p^{(i)}) - (1-y^{(i)}) \log(1-p^{(i)})$$

This is known as the Cross Entropy Loss

Let's check the intuition for this.

Let's say that we have a data point for which $y=1$. Then the loss collapses to $-\log(p)$. If our model predicts $p=1$ (Correct and Confident) the $-\log(1) = 0$, no loss. However, as $p \rightarrow 0$, $-\log(p) \rightarrow \infty$, massive loss. So massive penalty occurs for incorrect predictions.

Now, we theoretically have used MSE for our loss function, but CEL gives much stronger signals, helping it train more effectively.

Gradients

Unfortunately, logistic regression has no closed form solution, so we must use iterative methods. (It is also known that the CEL is convex, so only one global minimum).

First, sigmoid

$$\sigma(x) = \frac{e^x}{1+e^x} = e^x (1+e^x)^{-1}$$

$$\sigma'(x) = \sigma(x) (1 - \sigma(x))$$

Check that by hand. This is very nice as the sigmoid derivative is defined in terms of itself.

So, if we let $z = w^T x$, we have

$$L(w) = -y \log(p) - (1-y) \log(1-p)$$

Applying the chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial p} \times \frac{\partial p}{\partial z} \times \frac{\partial z}{\partial w}$$

$$\frac{\partial L}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p}$$

$$\frac{\partial p}{\partial z} = \sigma(z)(1-\sigma(z)) = p(1-p)$$

$$\frac{\partial z}{\partial w} = x$$

So all together:

$$\frac{\partial L}{\partial w} = \left(-\frac{y}{p} + \frac{1-y}{1-p}\right) \cdot p(1-p) \cdot x$$

$$= (-y(1-p) + (1-y)p)x$$

$$= (-y + py + p - py)x$$

$$= (p-y)x$$

This is very clean. So:

$$\nabla_w L = (\sigma(w^T x) - y) \cdot x$$

Summing over the whole data set

$$\nabla_w L = \frac{1}{N} \sum_i (\sigma(w^T x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

$$= \frac{1}{N} X^T (\sigma(Xw) - Y)$$

Where we have inputs X , outputs Y and σ is computed elementwise.

So the gradient descent update is:

$$w \leftarrow w - \frac{\alpha}{N} X^T (\sigma(Xw) - Y)$$

Where $p = \sigma(Xw)$ is computed every step.

Multiclass Classification: Softmax Regression

When $y \in \{1, 2, \dots, k\}$, we generalise logistic regression to softmax regression.

Instead of a single weight vector w , we have k weight vectors $w_1, \dots, w_k$, one for each class. We stack them into the rows of a weights matrix:

$$W \in \mathbb{R}^{k \times d}$$

The model computes k logits:

$$z_k = w_k^T x \quad \text{for } k=1, \dots, k$$

Then converts them into probabilities via softmax:

$$p(y=k|x) = \frac{e^{z_k}}{\sum_j e^{z_j}} = \text{Softmax}(z_k)$$

Softmax is the generalisation of sigmoid to higher dimensions. It has a few key properties:

- Softmax is invariant to adding a constant to all logits.

$$\text{Softmax}(z+c) = \text{Softmax}(z)$$

- This property is what makes softmax for $k=2$ equal to sigmoid

For $k=2$, softmax gives two weight vectors w_1 and w_2 , producing logits $z_1 = w_1^T x$, $z_2 = w_2^T x$. The softmax probability for class 1 equals:

$$p(y=1) = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}$$

We can use the invariance property to subtract any constant without changing the output. We subtract z_2 from both:

$$= \frac{e^{(z_1 - z_2)}}{e^{(z_1 - z_2)} + e^{(z_2 - z_2)}} \\ = \frac{e^{z_1 - z_2}}{1 + e^{z_1 - z_2}}$$

Let $u = z_1 - z_2 = (w_1 - w_2)^T x$. Then:

$$= \frac{e^u}{1 + e^u}$$

$$= \sigma(u)$$

$$= \sigma((w_1 - w_2)^T x)$$

Now, we only care about the difference between w_1 and w_2 , which means w_1 and w_2 can take any value they like, so long as the difference is correct. Thus, by convention, we always set $w_2 = 0$, so we only ever need to calculate w_1 . So we have

$$\sigma(w^T x) \quad \text{for } k=2.$$

There are some important practical reasons why we do this as well. If we let w_1 and w_2 take any value, gradient descent can drift along the direction where w_1 and w_2 grow together since this has no effect on the loss.

Additionally, the solution would not be unique, meaning if we trained the same model twice with different initialisations, you'd get different w_1 and w_2 but identical predictions. This makes the weights uninterpretable - they cannot be compared across runs.

- The softmax exhibits "winner-take-all" behaviour. If one logit is much larger than the others, its probability approaches 1.

- Numerically, if we have a very large z_k , like $z_k = 1000$, then e^{1000} is a huge number, which can't be stored in a 64-bit number. This causes overflow, the computer stores it as infinity, and all the arithmetic breaks. The fix for this is to shift all the logits down by the largest one.

$$\text{softmax}(z - z_{\max})$$

This gives the same result, but is numerically stable. After subtracting the maximum, the largest logit becomes 0, so $e^0 = 1$ - no overflow. All other logits are negative, so their exponentials are in $(0, 1]$. This is called the log-sum-exp trick. (for reasons that will become clear later)

Note: You might worry about underflow instead after performing this trick. But this is fine, since e^{-1000} (for example) ≈ 0 to machine precision anyway.

It would correspond to a vanishingly small probability, so 0 is a good approximation for it anyway.

Cross Entropy Loss for Multiclass

The derivation follows the same logic as the binary case.

The generalisation of the Bernoulli to K classes is the Categorical Distribution.

Instead of a single parameter p , we have K probabilities $p_1, p_2, \dots, p_K$ that all add to 1. For a single observation $y \in \{1, \dots, K\}$, the probability of observing class k is simply p_k . Just like in the binary case, we need a way to write this as one expression. We use one-hot encoding, we use a vector $y \in \{0, 1\}^K$ where the k th element of y is 1 if the true class is k , and 0 everywhere else. Then:

$$P(y) = p_1^{y_1} \cdot p_2^{y_2} \cdot \dots \cdot p_K^{y_K} \\ = \prod_k p_k^{y_k}$$

So we see that, since there is only one $y_k = 1$ and the rest are all 0, all terms are equal to 1 except the true class, leaving $P_{\text{true class}}$.

To get the likelihood over the entire dataset, assuming n independent observations, the joint likelihood is:

$$L(W) = \prod_i \prod_k (p_k^{(i)})^{y_k^{(i)}}$$

we compute softmax vector for all probabilities, then select the k th one

Where $p_k^{(i)} = \text{softmax}(Wx^{(i)})_k$ is the model's prediction for class k on example i .

We take the negative log likelihood:

$$L(W) = -\frac{1}{n} \sum_i \sum_k y_k^{(i)} \log(p_k^{(i)})$$

But recall that there is only one $y_k = 1$, the rest are all zero, so the inner sum simplifies:

$$L(W) = -\frac{1}{n} \sum_i \log(P(y = y^{(i)} | x^{(i)}))$$

← $P_{\text{true class}}$

This is the average negative log likelihood assigned to the correct class across all examples. Loss is low when model consistently assigns high probability to the true class, high otherwise.

Regularisation for Logistic Regression

The same techniques from linear regression apply directly, with the same motivation: reduce variance at the cost of a small increase in bias.

- L_2 (Ridge): add $\lambda \|W\|_2^2$
- L_1 (Lasso): add $\lambda \|W\|_1$

In practice, we always need to regularise for logistic regression. Let's think about why:

We call the input data linearly separable if there exists a hyperplane that perfectly separates the classes. In the binary case, every class 1 point on one side, every class 0 point on the other. Formally, it means there exists some w such that

$$w^T x^{(i)} > 0 \text{ for all } y^{(i)} = 1 \\ w^T x^{(i)} < 0 \text{ for all } y^{(i)} = 0$$

From the cross-entropy loss:

$$\text{If } y = 1, \text{ loss} = -\log(\sigma(w^T x))$$

$$\text{If } y = 0, \text{ loss} = -\log(1 - \sigma(w^T x))$$

For the loss to be zero, we need

$$\sigma(w^T x) = 1 \text{ when } y = 1$$

$$\sigma(w^T x) = 0 \text{ when } y = 0$$

but $\sigma(z) = 1$ only when $z \rightarrow +\infty$
and $\sigma(z) = 0$ only when $z \rightarrow -\infty$
It can only approach asymptotically.

When the data is linearly separable, there already exists a correct hyperplane, call it w^* . Consider scaling it: $10w^*$, $100w^*$, $1000w^*$ each still separates the data perfectly, with the logit larger in magnitude.

But larger logits just push the sigmoid output closer to 0 and 1, making the model more and more confident (the model assigns a higher and higher probability of the data belonging to that class, when in reality there might be more uncertainty).
So the logits keep increasing and increasing, ever driving the loss down.

Adding $\lambda \|w\|^2$ (L2 regularisation) penalises huge weights. The optimiser stops scaling w up and settles at a finite solution.

If we never regularised, gradient descent would never stop. We would have to stop it manually after an arbitrary amount of iterations. The confidence also is an issue. In reality, a point close to the decision boundary should be uncertain. But an overconfident model would give wildly miscalibrated probabilities. It might say it is 99.9999% sure it belongs to class 1, for example. This matters a lot in practice, as in fields like medicine, finance or others, the probability itself is used in decision-making.

Evaluation For Classification

The Confusion Matrix

For k classes, we construct a $k \times k$ matrix where entry (i, j) is the

number of examples with true class i , that were given a prediction of j .
Thus, the diagonal entries are correct predictions, everything off diagonal is an error. There are a few metrics we can get from this:

- **Accuracy** - fraction of correct predictions. This can be misleading when classes are imbalanced. If the model always picks the class for which most of the data points belong to, it will have a high accuracy.

- **Per-class Precision** - What fraction of the positive predictions were true positives? We temporarily treat class k as the positive class, all others as negative. We calculate

True Positives

True Positives + False Positives

Relevant when false positives are costly, e.g. spam filtering. You don't want to block legitimate email.

- **Per-class Recall** - Of the actual positives, how many did we catch?

True Positives

True Positives + False Negatives

Relevant when false negatives are costly. e.g. Cancer Screening - you don't want to miss a tumour.

- **F1 Score** - Harmonic mean of precision and recall.

$2 (\text{Precision} \times \text{Recall})$

Precision + Recall

Gives a single number that balances both

